Arabic Function Forge at AISA-ArabicFC: Copy, Don't Interpret Training-Free Arabic Function Calling

Anfal Alshehri
KAUST, Thuwal, Saudi Arabia

Jumanah Aljubayri
Tahakom, Riyadh, Saudi Arabia

Abstract

Arabic Function Forge is our submission to AISA-ArabicFC 2026. Our system is training-free: it uses a single off-the-shelf open-weights model, prompted once per request, with no parameter updates. It placed fifth on Track A, fourth on Track B, and sixth on Track C. Our central observation is that function selection is nearly saturated on this benchmark while argument extraction is not: we reach 0.9893 function accuracy but only 0.8686 argument exact match, so 94.9% of the remaining headroom lies in arguments. The dominant failure is not misunderstanding but over-interpretation: the model translates values, converts dates to ISO form, and adds fields the user never provided. Our system is designed to suppress this behaviour by preserving argument values as written. We further find that the dialect penalty is concentrated rather than graded, with most of the drop occurring on Maghrebi.

1 Introduction

With the recent advances in agentic workflows (Nacar et al., 2026c), function calling has become increasingly important. Function calling allows a language model to turn a natural language request into a structured call that invokes a function to do some job. This requires two main jobs: choosing the right function and extracting the right arguments for it. Both have to be correct. A call with the right function but the wrong arguments is still a failed call.

However, most current systems are developed and evaluated in English. The recent survey of agentic tool use (Hu et al., 2026) covers different approaches to function calling, but does not address language diversity or non-English evaluation. For Arabic, this matters because performance drops by 5–10% when models are prompted in Arabic, whether the tool descriptions are written in Arabic or English (Kubrak et al., 2026). Arabic also carries cultural and religious contexts that are not always captured by English-based systems (Al-Khalifa et al., 2025), and the problem becomes harder with dialects, where the value to extract is whatever the user actually wrote.

Most existing approaches solve function calling through some form of training. Toolformer (Schick et al., 2023) learns tool use self-supervised, ToolLLM (Qin et al., 2024) instruction-tunes over 16,000+ APIs, and Gorilla (Patil et al., 2024) uses fine-tuning for API calls. For Arabic, Nacar et al. (2026b) show that data-centric fine-tuning can reduce parse failures from 87% to under 1%. The problem is that these approaches depend on training, and changes to the available tools may require additional data or model updates. We instead take a training-free approach where tools are provided to the model at inference time.

Our system is based on three main decisions: (i) providing only the four candidate tools associated with each request rather than the full 27-tool registry; (ii) using the reference clock provided with each example to handle relative time expressions; and (iii) using an Arabic system prompt built from our development error analysis, with one main idea: copy the user's values as written instead of interpreting them.

Without any parameter updates, our system placed fifth on Track A, fourth on Track B, and sixth on Track C (Table 1). Our results show that function selection is close to saturated while argument extraction remains the main source of error. We also find that the dialect gap is not equal across Arabic varieties, but is concentrated mainly in Maghrebi. We analyse both findings in §4.

2 Related Work

Function calling is not a new problem. Before LLMs, a similar problem was studied as *intent detection* and *slot filling*: deciding what the user wants and extracting the values needed for that re-

quest. This is closely related to choosing a function and filling its arguments. Weld et al. (2022) survey more than a decade of work on these tasks, while Chen et al. (2019) use BERT to jointly predict intents and slots. These systems, however, are built around a fixed schema, where adding new intents or slots can require new labelled data and retraining. MASSIVE (FitzGerald et al., 2023) extended this setting to 51 languages, including Saudi Arabic (`ar-SA`), but still uses a fixed inventory of 60 intents and 55 slots.

LLMs changed this by allowing tool schemas to be provided directly in the prompt. Hu et al. (2026) divide current approaches into prompting, supervised tool learning, and reward-driven tool learning; our system belongs to the first. Previous systems learn tool use through self-supervised learning (Schick et al., 2023), instruction tuning (Qin et al., 2024), or fine-tuning with retrieval (Patil et al., 2024). ToolkenGPT (Hao et al., 2023) keeps the language model frozen but learns an embedding for each tool. In contrast, our system requires no schema-specific parameter updates: a new tool is provided as a JSON schema at inference time.[1]

Function calling is commonly evaluated using API-Bank (Li et al., 2023) and BFCL (Patil et al., 2025). AISA-ArabicFC differs in that requests are Arabic and arguments are evaluated using strict key–value exact match. For Arabic, Kubrak et al. (2026) show that function-calling accuracy drops, while Nacar et al. (2026b) introduce the corpus underlying this shared task, described by Nacar et al. (2026a). These works establish that Arabic function calling degrades, but do not localise where argument extraction fails or separate extraction errors from dialect effects. This is the gap we study.

3 Methodology

We use the open-weights Qwen3.5-122B-A10B model (Qwen Team, 2026), served locally with vLLM (Kwon et al., 2023), without fine-tuning or retrieval. The labelled data are used only for error analysis and prompt design; no model parameters are updated.

For each AISA-ArabicFC request (Nacar et al., 2026a,b), AISA-ArabicFC provides four candidate tools in `tools_sampled`, which we pass directly to the model rather than the full 27-tool registry . Before inference, we sanitise the schemas by removing null properties and empty enums and updating the required fields. We also provide the reference clock included with each example so relative time expressions are resolved against the intended date rather than the current time.

The system prompt is written in Arabic and was built from our development error analysis. Its main instruction is to copy user values as written rather than translate, normalise, or infer them. We also add rules for the most common errors, including dates, the Arabic definite article, lists, optional fields, and tool-specific argument conventions.

We perform one generation per request with automatic tool selection, temperature 0.6, and Qwen thinking enabled. The function call is used for Track A and the Arabic reasoning from the same generation for Track B, with no second generation or translation step. Null and empty arguments are removed from the final call; no other post-processing is applied.

Track	FnAcc	ArgEM	Overall
A (5th)	0.9893	0.8686	**0.9169**
B (4th)	0.9893	0.8686	**0.9311**
C (6th)	—	—	**0.8847**

Table 1: Blind-test results and final leaderboard ranks. Track B has a ThinkRate of 1.0000, while Track C reports the macro average across five dialects.

4 Evaluation

We follow the official shared-task protocol (Nacar et al., 2026a) on 1,125 blind test examples, with no manual selection or modification of the predictions. Function Accuracy (FnAcc) measures correct function selection, while Argument Exact Match (ArgEM) requires the full argument dictionary to match exactly with no partial credit. Track A weights them as $0.40 \cdot \mathrm{FnAcc} + 0.60 \cdot \mathrm{ArgEM}$, Track B additionally evaluates the Arabic reasoning trace, and Track C evaluates dialect robustness.

The main remaining problem is argument extraction. FnAcc reaches 0.9893, while ArgEM is 0.8686. Under the Track A weighting, perfect function selection would improve Overall by only 0.0043, compared with 0.0788 for perfect argument extraction. This means that 94.9% of the remaining headroom lies in arguments. The dialect results also show that the degradation is not uniform: MSA and Egyptian score 0.933, Gulf

[1] Training-free is not cost-free. Serving a 122B-parameter Mixture-of-Experts model requires substantial GPU memory; our claim concerns maintenance as the tool registry changes, not total compute.

0.890, Levantine 0.880, and Maghrebi 0.788. The 0.145 best–worst gap is therefore mainly driven by Maghrebi, showing that the remaining dialect problem is concentrated rather than shared equally across Arabic varieties.

5 Conclusion

We presented a training-free approach for Arabic function calling using an open-weights model and prompt-based inference. Without fine-tuning, our system achieved an Overall score of 0.9169 on Track A, with 0.9893 function accuracy and 0.8686 argument exact match. This shows that the main remaining challenge is argument extraction rather than function selection. We also find that dialect degradation is concentrated mainly in Maghrebi. These results show that competitive Arabic function calling can be achieved without parameter updates.

Limitations

Although no model parameters are updated, our prompt was designed using labelled development data and therefore still uses supervision. Some prompt rules are also specific to the 27 tools and may not transfer directly to a new registry. Finally, our dialect results, particularly for Maghrebi, are based on a smaller portion of the data and should be interpreted with caution. Our results therefore show the potential of training-free function calling, rather than that it is more accurate than supervised approaches.

Acknowledgments

The authors declare that there is no conflict of interest. This work represents the authors' independent efforts and received no support or funding from any external organization. We thank Tuwaiq Academy and the AISA-ArabicFC organizers for providing the dataset, baseline model, and evaluation framework for this shared task.

References

Sarah Al-Khalifa, Nadir Durrani, Hend Al-Khalifa, and Firoj Alam. 2025. The landscape of Arabic large language models (ALLMs): A new era for Arabic language technology. *arXiv preprint arXiv:2506.01340*.

Qian Chen, Zhu Zhuo, and Wen Wang. 2019. BERT for joint intent classification and slot filling. *arXiv preprint arXiv:1902.10909*.

Jack FitzGerald, Christopher Hench, Charith Peris, Scott Mackie, Kay Rottmann, Ana Sanchez, Aaron Nash, Liam Urbach, Vishesh Kakarala, Richa Singh, Swetha Ranganath, Laurie Crist, Misha Britan, Wouter Leeuwis, Gokhan Tur, and Prem Natarajan. 2023. MASSIVE: A 1M-example multilingual natural language understanding dataset with 51 typologically-diverse languages. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4277–4302.

Shibo Hao, Tianyang Liu, Zhen Wang, and Zhiting Hu. 2023. ToolkenGPT: Augmenting frozen language models with massive tools via tool embeddings. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Jinchao Hu, Meizhi Zhong, Kehai Chen, Xuefeng Bai, and Min Zhang. 2026. Agentic tool use in large language models: A survey. *arXiv preprint arXiv:2604.00835*.

Konstantin Kubrak, Ahmed El-Moselhy, Ammar Alsulami, Remaz Altuwaim, Hassan Ismail Fawaz, and Faisal Alsaby. 2026. Arabic prompts with English tools: A benchmark. *arXiv preprint arXiv:2601.05101*.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*.

Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. 2023. API-Bank: A comprehensive benchmark for tool-augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3102–3116.

Omer Nacar, Mohammed Al Khalifa, and Saeed Alzaharani. 2026a. AISA-ArabicFC: The first shared task on Arabic function calling for agentic AI systems. In *Proceedings of The Fourth Arabic Natural Language Processing Conference: Shared Tasks*, Budapest, Hungary. Association for Computational Linguistics.

Omer Nacar, Deema Alquffari, Saleh Alsharideh, Adeem AlOtaibi, Abdulaziz Alabdulkarim, Leen Alhazmi, Nada Alomar, Wareef Alzubaidi, Nada Alsultan, Ahmed Alrabghi, et al. 2026b. From language to action in Arabic: Reliable structured tool calling via data-centric fine-tuning. *arXiv preprint arXiv:2603.16901*.

Omer Nacar, A. Deema, and A. Mohammed. 2026c. AISA: A unified architecture for agentic AI systems.

Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. 2025. The Berkeley function

calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *PMLR*.

Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2024. Gorilla: Large language model connected with massive APIs. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *International Conference on Learning Representations (ICLR)*.

Qwen Team. 2026. Qwen3.5-122B-A10B. https://huggingface.co/Qwen/Qwen3.5-122B-A10B.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Henry Weld, Xiaoqi Huang, Siqu Long, Josiah Poon, and Soyeon Caren Han. 2022. A survey of joint intent detection and slot filling models in natural language understanding. *ACM Computing Surveys*, 55(8):1–38.